

Anotações — Aula 4: Ensemble Learning

(Aprendizado em Conjunto)

Disciplina: Machine Learning — Pós-Graduação Ciência de Dados e Big Data / Estatística

Professor: Murilo Afonso Robiati Bigoto (PUC Minas)

Dataset: `vinhos.csv` — qualidade físico-química de vinhos tintos

Notebook: `Ensemble_Learning_Aula.ipynb`

Frase-chave: “Os algoritmos que vimos hoje são o estado da arte em ML. Quando colocados em uso prático, eles fazem chover — são os de maior poder preditivo.”

Contexto da Aula

Esta aula fecha o ciclo de algoritmos supervisionados, avançando de **single learners** (um modelo treinado sozinho) para **ensemble learners** (múltiplos modelos colaborando). É o paradigma de maior poder preditivo em ML clássico — XGBoost, LightGBM e Random Forest dominam competições e produção até hoje.

Dois problemas trabalhados com o mesmo dataset:

- **Regressão:** prever a nota de qualidade do vinho (0–10)
 - **Classificação:** classificar o vinho em *Ruim* (≤ 5), *Médio* (6) ou *Bom* (≥ 7)
-

Imports da Aula

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Pré-processamento
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
```

```
# Modelos base
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor
from sklearn.linear_model import LogisticRegression, Ridge
from sklearn.neighbors import KNeighborsClassifier

# Ensembles — sklearn
from sklearn.ensemble import (
    BaggingClassifier, BaggingRegressor,
    RandomForestClassifier, RandomForestRegressor,
    AdaBoostClassifier, AdaBoostRegressor,
    GradientBoostingClassifier, GradientBoostingRegressor,
    StackingClassifier, StackingRegressor,
    VotingClassifier, VotingRegressor,
    ExtraTreesClassifier
)

# Ensembles modernos (instalar antes se necessário)
from xgboost import XGBClassifier, XGBRegressor
from lightgbm import LGBMClassifier, LGBMRegressor

# Métricas
from sklearn.metrics import (accuracy_score, classification_report,
                             confusion_matrix, mean_squared_error,
                             mean_absolute_error, r2_score)

SEED = 42
```

Dataset: Qualidade de Vinhos Tintos

```
df = pd.read_csv('vinhos.csv')
```

Coluna	Descrição
acidez_fixa	Ácidos não voláteis (g/L)
acidez_volatil	Ácido acético — vinagre (g/L)

acido_citrico	Frescor e sabor (g/L)
acucar_residual	Açúcar após fermentação (g/L)
cloretos	Teor de sal (g/L)
dioxido_enxofre_livre	SO ₂ livre — conservante (mg/L)
dioxido_enxofre_total	SO ₂ total (mg/L)
densidade	Densidade da amostra (g/cm ³)
ph	Acidez geral
sulfatos	Conservante e antimicrobiano (g/L)
alcool	Teor alcoólico (% vol)
qualidade	TARGET — Nota do enólogo (0–10)

Pré-processamento

```
df_clean = df.copy()

# 1. Remover duplicatas
df_clean = df_clean.drop_duplicates()

# 2. Tratar outliers em cloretos (critério IQR tolerante: Q3 + 3×IQR)
col = 'cloretos'
Q1, Q3 = df_clean[col].quantile([0.25, 0.75])
limite_sup = Q3 + 3 * (Q3 - Q1)
df_clean[col] = df_clean[col].clip(upper=limite_sup)

# 3. Imputar missings com mediana
for col in ['acido_citrico', 'sulfatos', 'acucar_residual']:
    df_clean[col] = df_clean[col].fillna(df_clean[col].median())

# 4. One-Hot na variável categórica 'lote'
df_clean = pd.get_dummies(df_clean, columns=['lote'], drop_first=False)

# 5. Criar target de classificação
def categorizar_qualidade(nota):
    if nota <= 5: return 'Ruim'
```

```
elif nota == 6: return 'Médio'
else: return 'Bom'
```

```
df_clean['categoria'] = df_clean['qualidade'].apply(categorizar_qualidade)
```

Por que $Q3 + 3 \times IQR$ em vez do padrão $1.5 \times IQR$? No contexto de vinhos, valores extremos de claretos podem ser legítimos (vinhos de regiões específicas). O critério mais tolerante evita remover observações válidas. Sempre ajuste o critério ao domínio do problema.

O Que é Ensemble Learning?

Definição: Treinar múltiplos modelos (chamados de *weak learners*) e combinar suas previsões para formar um *strong learner* mais preciso e robusto.

Analogia (Teorema do Júri de Condorcet): Imagine um júri de 21 juízes, cada um com 60% de chance de votar corretamente. Se votarem de forma independente, a probabilidade de a maioria acertar se aproxima de 100%. Isso é o princípio matemático do Ensemble — quanto mais “juízes” independentes e razoavelmente bons, melhor a decisão coletiva.

Vantagens:

- Melhor desempenho preditivo
- Redução de variância e/ou viés
- Maior resistência a outliers

Desvantagens:

- Requerem mais dados para treino
- Computacionalmente mais caros
- Mais difíceis de interpretar (caixa preta)

As 3 premissas do Ensemble

Premissa	O que significa
Diversidade	Os modelos devem cometer erros <i>diferentes</i> entre si
Redução de Variância	Erros individuais se compensam quando combinados

Combinação	Estratégia de agregar as saídas (média, votação, meta-modelo)
-------------------	---

PARTE 1 — Bagging (Bootstrap Aggregating)

Conceito

Treina múltiplos modelos **em paralelo e de forma independente**, cada um em um subconjunto amostrado **com reposição** (bootstrap) dos dados de treino.

Dataset original (N amostras)

B₁ B₂ B₃ B₄ bootstraps (~63% únicos cada)

M₁ M₂ M₃ M₄ modelos independentes

Votação (clf) / Média (reg)

*O que é amostragem com reposição? É como sortear nomes de uma urna mas colocar o papel de volta antes do próximo sorteio. Cada bootstrap tem ~63,2% de amostras únicas originais — os 36,8% restantes são os **Out-Of-Bag (OOB)**, usados para validação interna sem precisar de um conjunto de validação separado.*

Código — Bagging Classificação

```
# Árvore simples como baseline
arvore_base = DecisionTreeClassifier(random_state=SEED)
arvore_base.fit(X_train_cs, y_train_c)
acc_arvore = accuracy_score(y_test_c, arvore_base.predict(X_test_cs))
print(f'Árvore simples (baseline): {acc_arvore:.4f}')

# BaggingClassifier
bagging_clf = BaggingClassifier(
    estimator=DecisionTreeClassifier(random_state=SEED),
    n_estimators=100, # número de modelos base
    max_samples=0.8, # 80% das amostras por bootstrap
```

```
max_features=0.8, # 80% das features por modelo
random_state=SEED,
n_jobs=-1 # paraleliza em todos os cores
)
bagging_clf.fit(X_train_cs, y_train_c)
acc_bag = accuracy_score(y_test_c, bagging_clf.predict(X_test_cs))
print(f'Bagging: {acc_bag:.4f}')
```

Random Forest — O Estado da Arte do Bagging

O **Random Forest** é um Bagging de Árvores de Decisão com um ingrediente extra: em cada divisão de nó, apenas um **subconjunto aleatório de features** é considerado. Isso força diversidade entre as árvores e é o segredo do seu sucesso.

```
# Classificação
rf_clf = RandomForestClassifier(
    n_estimators=100,
    max_features='sqrt', # raiz quadrada do total de features em cada split
    random_state=SEED,
    n_jobs=-1
)
rf_clf.fit(X_train_cs, y_train_c)
acc_rf = accuracy_score(y_test_c, rf_clf.predict(X_test_cs))
print(f'Random Forest: {acc_rf:.4f}')

# Regressão
rf_reg = RandomForestRegressor(n_estimators=100, random_state=SEED, n_jobs=-1)
rf_reg.fit(X_train_s, y_train_r)
pred_rf = rf_reg.predict(X_test_s)
print(f'RMSE: {np.sqrt(mean_squared_error(y_test_r, pred_rf)):.4f}')
print(f'R² : {r2_score(y_test_r, pred_rf):.4f}')
```

Feature Importance no Random Forest

```
importancias = pd.Series(
    rf_clf.feature_importances_,
    index=feature_cols
).sort_values(ascending=True)
```

```
importancias.plot(kind='barh', figsize=(10, 6))
plt.title('Importância das Features — Random Forest')
plt.xlabel('Importância (Gini)')
plt.tight_layout()
plt.show()
```

A importância de cada feature no Random Forest é calculada como a **média da redução de impureza Gini** que aquela variável provoca em todas as árvores. Features com maior importância têm mais poder de separar as classes.

PARTE 2 — Boosting

Conceito

Ao contrário do Bagging (paralelo), o Boosting é **sequencial**: cada novo modelo foca nos erros cometidos pelo conjunto anterior.

M_1 erros(M_1) M_2 erros(M_1+M_2) M_3 ... M_n
Previsão final = combinação ponderada de $M_1 + M_2 + \dots + M_n$

- **Bagging** reduz **variância** (combate overfitting)
- **Boosting** reduz **viés** (combate underfitting, melhora modelos fracos)

Algoritmo	Mecanismo principal	Destaque
AdaBoost	Pondera exemplos mal classificados	Simples e interpretável
Gradient Boosting	Otimiza o gradiente da função de perda	Robusto e flexível
XGBoost	GBM + regularização L1/L2 + poda	Vencedor de competições
LightGBM	Cresce por folha (leaf-wise)	Muito mais veloz

AdaBoost

```
ada_clf = AdaBoostClassifier(  
    estimator=DecisionTreeClassifier(max_depth=2, random_state=SEED), # weak learner
```

```

n_estimators=200,
learning_rate=0.5, # peso de cada estimador na combinação final
random_state=SEED
)
ada_clf.fit(X_train_cs, y_train_c)
acc_ada = accuracy_score(y_test_c, ada_clf.predict(X_test_cs))
print(f'AdaBoost: {acc_ada:.4f}')

# Curva de evolução da acurácia por número de estimadores
accs = [accuracy_score(y_test_c, p) for p in ada_clf.staged_predict(X_test_cs)]

plt.figure(figsize=(9, 4))
plt.plot(accs, color='darkorange', lw=2, label='AdaBoost')
plt.axhline(acc_arvore, ls='--', color='gray', label=f'Árvore simples ({acc_arvore:.3f})')
plt.xlabel('Número de Estimadores')
plt.ylabel('Acurácia')
plt.title('AdaBoost — Evolução da Acurácia')
plt.legend()
plt.show()

```

***Lição do gráfico AdaBoost:** O professor destacou que quando a curva fica **ruidosa** (oscilando bastante) após um platô, é sinal de que adicionar mais árvores está causando **overfitting**. Parar no início do ruído é a estratégia certa.*

Gradient Boosting, XGBoost e LightGBM

```

import time

# Gradient Boosting (sklearn)
gb_clf = GradientBoostingClassifier(
    n_estimators=200,
    learning_rate=0.05, # quanto cada árvore contribui para corrigir o erro
    max_depth=4,        # profundidade de cada árvore base (geralmente 3-5)
    subsample=0.8,      # fração dos dados usada em cada etapa
    random_state=SEED
)
t0 = time.time(); gb_clf.fit(X_train_cs, y_train_c); t_gb = time.time() - t0

# XGBoost
xgb_clf = XGBClassifier(

```



```

n_estimators=200,
learning_rate=0.05,
max_depth=4,
subsample=0.8,
colsample_bytree=0.8, # fração de features por árvore (como o max_features do RF)
eval_metric='mlogloss',
random_state=SEED, verbosity=0
)
t0 = time.time(); xgb_clf.fit(X_train_cs, y_train_c); t_xgb = time.time() - t0

# LightGBM
lgbm_clf = LGBMClassifier(
    n_estimators=200,
    learning_rate=0.05,
    max_depth=4,
    subsample=0.8,
    random_state=SEED, verbose=-1
)
t0 = time.time(); lgbm_clf.fit(X_train_cs, y_train_c); t_lgbm = time.time() - t0

print(f'GBM — Acurácia: {accuracy_score(y_test_c, gb_clf.predict(X_test_cs)):.4f} | Tempo: {t_gb:.4f}s')
print(f'XGB — Acurácia: {accuracy_score(y_test_c, xgb_clf.predict(X_test_cs)):.4f} | Tempo: {t_xgb:.4f}s')
print(f'LGBM — Acurácia: {accuracy_score(y_test_c, lgbm_clf.predict(X_test_cs)):.4f} | Tempo: {t_lgbm:.4f}s')

```

Resultado do professor em aula:

Algoritmo	Tempo de treino
Gradient Boosting	~8 segundos
XGBoost	~1 segundo
LightGBM	~0,3 segundos

*Por que **LightGBM** é mais rápido? Algoritmos tradicionais de boosting (GBM) crescem **árvore por nível** (level-wise). O **LightGBM** cresce **por folha** (leaf-wise) — ele sempre divide a folha que traz o maior ganho. Isso reduz drasticamente o número de operações e a memória usada.*

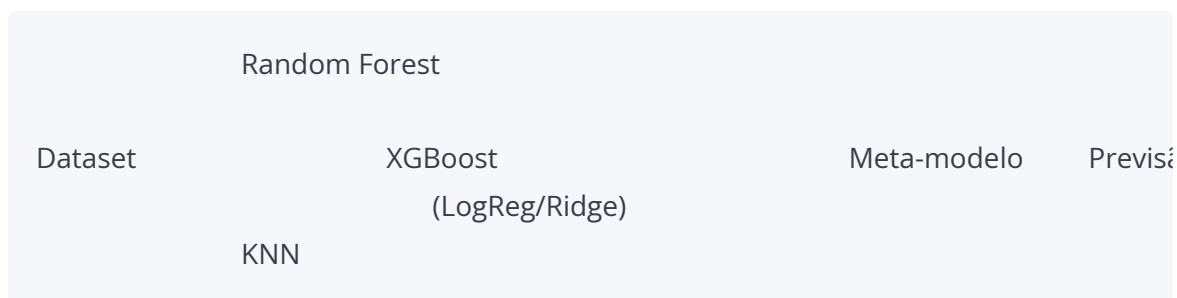
Hiperparâmetros críticos do Boosting

Parâmetro	O que controla	Dica prática
n_estimators	Número de árvores	Monitore a curva de aprendizado para não overfittar
learning_rate	Contribuição de cada árvore	Menor = mais robusto, mais lento; tente 0.05–0.1
max_depth	Profundidade de cada árvore base	3–5 para boosting (árvores rasas!)
subsample	% de dados por iteração	0.7–0.9 adiciona estocasticidade saudável
colsample_bytree	% de features por árvore (XGB/LGBM)	Similar ao max_features do Random Forest

PARTE 3 — Stacking

Conceito

O **Stacking** treina modelos de **Nível 0** (base learners) e usa as *previsões deles* como novas features para treinar um **meta-modelo** (Nível 1).



Analogia: O Stacking é como uma equipe médica em que vários especialistas (cardiologista, neurologista, ortopedista) dão seus diagnósticos independentes, e um clínico geral (meta-modelo) integra todas as opiniões para dar o diagnóstico final — ponderando quem tem mais acerto em cada tipo de caso.

Código — Stacking Classificação

```
# Modelos base (nível 0)
estimadores_base = [
    ('rf', RandomForestClassifier(n_estimators=100, random_state=SEED, n_jobs=-1)),
```

```

('xgb', XGBClassifier(n_estimators=100, learning_rate=0.05, random_state=SEED,
                      use_label_encoder=False, eval_metric='mlogloss', verbosity=0)),
('knn', KNeighborsClassifier(n_neighbors=7, n_jobs=-1)),
]

# Meta-modelo (nível 1) — geralmente um modelo simples
meta_modelo = LogisticRegression(max_iter=500, random_state=SEED)

stacking_clf = StackingClassifier(
    estimators=estimadores_base,
    final_estimator=meta_modelo,
    cv=5,          # cross-validation para gerar as previsões OOF (out-of-fold)
    passthrough=False, # True = passa features originais ao meta-modelo também
    n_jobs=-1
)
stacking_clf.fit(X_train_cs, y_train_c)
acc_stk = accuracy_score(y_test_c, stacking_clf.predict(X_test_cs))
print(f'Stacking: {acc_stk:.4f}')

```

Código — Stacking Regressão

```

estimadores_base_r = [
    ('rf', RandomForestRegressor(n_estimators=100, random_state=SEED, n_jobs=-1)),
    ('xgb', XGBRegressor(n_estimators=100, learning_rate=0.05, random_state=SEED, verbosity=0)),
    ('gb', GradientBoostingRegressor(n_estimators=100, learning_rate=0.05, random_state=SEED, n_jobs=-1)),
]

meta_modelo_r = Ridge(alpha=1.0) # meta-modelo simples para regressão

stacking_reg = StackingRegressor(
    estimators=estimadores_base_r,
    final_estimator=meta_modelo_r,
    cv=5,
    n_jobs=-1
)
stacking_reg.fit(X_train_s, y_train_r)
pred_stk = stacking_reg.predict(X_test_s)
print(f'RMSE: {np.sqrt(mean_squared_error(y_test_r, pred_stk)):.4f}')
print(f'R² : {r2_score(y_test_r, pred_stk):.4f}')

```

Por que o meta-modelo costuma ser simples? O meta-modelo (LogReg, Ridge) é propositalmente simples para evitar overfitting nas previsões dos modelos base. Um meta-modelo complexo poderia memorizar os erros dos modelos base ao invés de aprender a combiná-los de forma generalista.

PARTE 4 — Voting

Conceito

Combina múltiplos modelos de **diferentes naturezas** por votação (classificação) ou média (regressão) — sem um meta-modelo de aprendizado.

- **Hard Voting:** a classe mais votada vence
- **Soft Voting:** média das probabilidades de cada classe (geralmente superior ao hard)

```
# Hard Voting — Classificação
voting_clf = VotingClassifier(
    estimators=[
        ('rf', RandomForestClassifier(n_estimators=100, random_state=SEED)),
        ('lgbm', LGBMClassifier(n_estimators=100, random_state=SEED, verbose=-1)),
        ('xgb', XGBClassifier(n_estimators=100, random_state=SEED, verbosity=0)),
    ],
    voting='hard' # ou 'soft' para média das probabilidades
)
voting_clf.fit(X_train_cs, y_train_c)
acc_vote = accuracy_score(y_test_c, voting_clf.predict(X_test_cs))
print(f'Voting (hard): {acc_vote:.4f}')

# Voting — Regressão (sempre faz média)
voting_reg = VotingRegressor(
    estimators=[
        ('rf', RandomForestRegressor(n_estimators=100, random_state=SEED)),
        ('xgb', XGBRegressor(n_estimators=100, random_state=SEED, verbosity=0)),
        ('gb', GradientBoostingRegressor(n_estimators=100, random_state=SEED)),
    ]
)
voting_reg.fit(X_train_s, y_train_r)
```

```
pred_vote = voting_reg.predict(X_test_s)
print(f'Voting Regressor — R²: {r2_score(y_test_r, pred_vote):.4f}')
```

Comparação Final dos Modelos

```
# Resumo de classificação
print('CLASSIFICAÇÃO — Acurácia:')
print(f' Árvore simples : {acc_arvore:.4f}   baseline')
print(f' Bagging       : {acc_bag:.4f}')
print(f' Random Forest   : {acc_rf:.4f}')
print(f' AdaBoost        : {acc_ada:.4f}')
print(f' Gradient Boost   : {acc_gb:.4f}')
print(f' XGBoost          : {acc_xgb:.4f}')
print(f' LightGBM         : {acc_lgbm:.4f}')
print(f' Stacking         : {acc_stk:.4f}')
print(f' Voting           : {acc_vote:.4f}')
```

Observação do professor: Random Forest e Stacking (que inclui RF como base) atingiram os melhores patamares (~74% de acurácia). XGBoost e LightGBM ficaram em ~70%. O professor destacou que todos os modelos estavam muito próximos — evidência do **teorema “No Free Lunch”** (não existe um algoritmo universalmente superior).

Mapa Mental dos Ensembles

ENSEMBLE LEARNING

BAGGING (paralelo — reduz variância)

BaggingClassifier / BaggingRegressor

Random Forest extra: subconjunto de features por split

ExtraTrees divisões completamente aleatórias

BOOSTING (sequencial — reduz viés)

AdaBoost pondera exemplos difíceis

Gradient Boosting otimiza gradiente da loss

XGBoost GBM + regularização L1/L2 + poda

LightGBM leaf-wise, muito mais veloz

CatBoost especialista em variáveis categóricas

STACKING (hierárquico — meta-modelo aprende a combinar)

Nível 0: modelos base de naturezas diversas

Nível 1: meta-modelo simples (LogReg, Ridge)

VOTING (democrático — sem aprendizado extra)

Hard: classe mais votada

Soft: média das probabilidades

Quando Usar Cada Estratégia?

Estratégia	Use quando...	Cuidado
Random Forest	Boa baseline em qualquer problema	Menos interpretável que árvore simples
XGBoost / LightGBM	Quer máxima performance, dados tabulares	Requer tuning cuidadoso de hiperparâmetros
AdaBoost	Quer algo mais simples e interpretável	Sensível a outliers e ruído
Stacking	Quer combinar pontos fortes de algoritmos diferentes	Complexo, pode overfittar no meta-modelo
Voting	Quer diversidade sem complexidade extra	Não aprende a ponderar os modelos

Pontos-Chave para Fixar

1. **Ensemble = “sabedoria das multidões”** — vários modelos medíocres combinados batem um modelo brilhante sozinho
2. **Bagging** paralelo reduz **variância** usa bootstrap (amostragem com reposição)
3. **Random Forest** = Bagging de árvores + subconjunto aleatório de features por nó
4. **Boosting** sequencial reduz **viés** cada modelo corrige os erros do anterior
5. **learning_rate** no **Boosting** controla quanto cada árvore contribui — menor = mais robusto, mais lento

6. **Ruído na curva AdaBoost** = sinal de overfitting com excesso de estimadores
 7. **LightGBM** é mais rápido porque cresce leaf-wise ao invés de level-wise
 8. **Stacking** usa o meta-modelo para *aprender* a combinar — por isso supera o Voting em geral
 9. **OOB (Out-Of-Bag)** = os ~37% de amostras não sorteadas em cada bootstrap usados como validação interna gratuita no Bagging
 10. **No Free Lunch**: não existe algoritmo universalmente superior — o contexto e os dados determinam o melhor modelo
-

Referências

- [Scikit-learn — Ensemble Methods](#)
- [XGBoost Documentation](#)
- [LightGBM Documentation](#)
- Géron, A. *Hands-On Machine Learning* — Cap. 7 (Ensemble Learning and Random Forests)
- Slides da Aula 4 — `Aula4.pptx`
- Notebook da Aula 4 — `Ensemble_Learning_Aula.ipynb`